

Lab Meeting

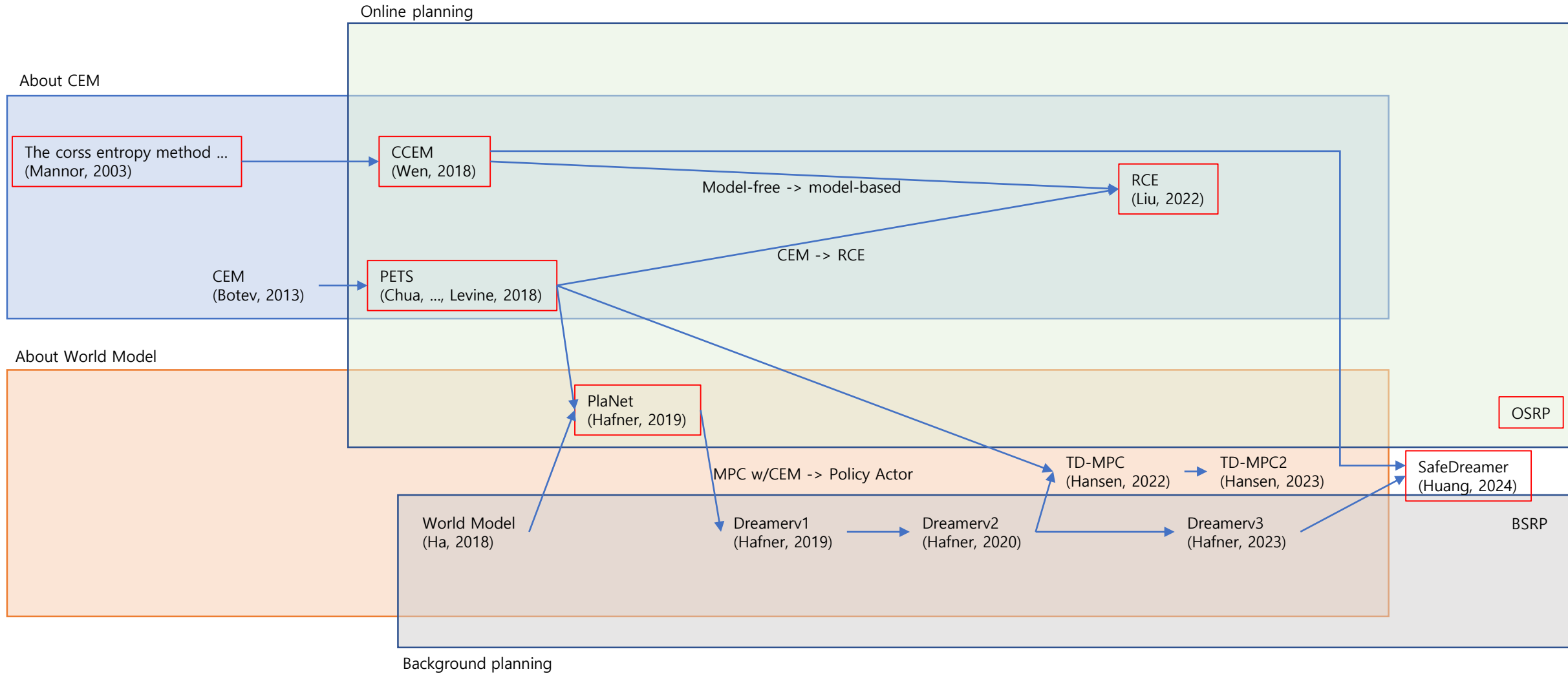
2024/7/16

Hyeongchan Ham

Contents

- Taxonomy
 - World Model and MPC
- Paper Review
 - Constrained Cross Entropy Method
- Current Works
 - Applied KAIST EE
 - Group Meeting Presentation
- Future Plan
 - OOD Autonomous Driving

Taxonomy



Constrained Cross-Entropy Method for Safe Reinforcement Learning

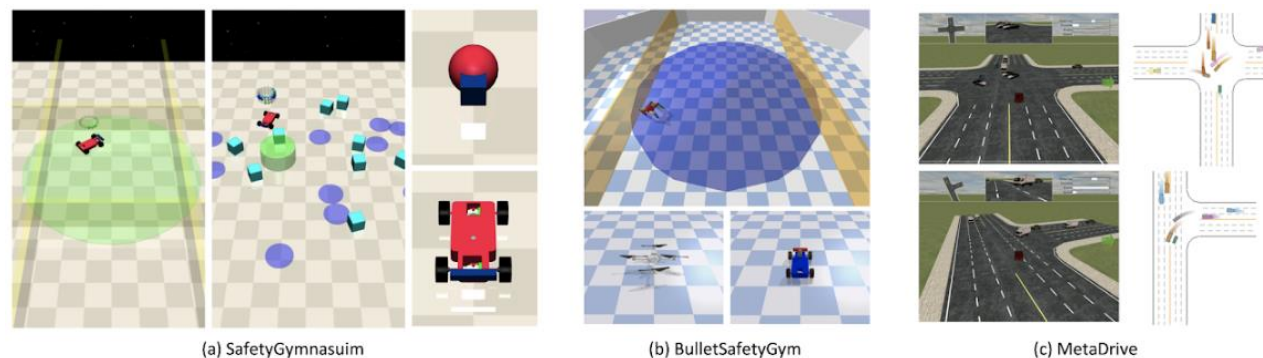
Min Wen, Ufuk Topcu

NIPS 2018

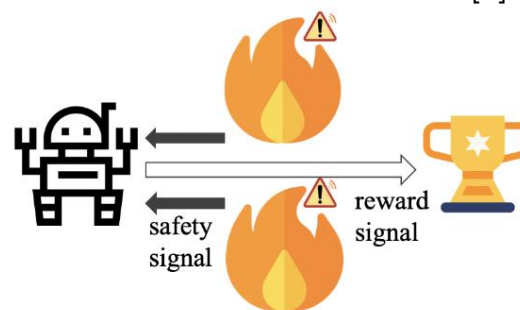
Introduction

- Safe Reinforcement Learning Task

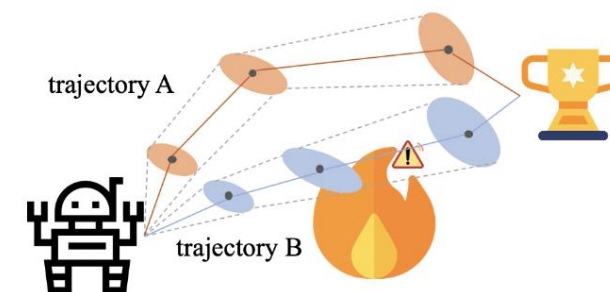
[1] safe RL tasks



[2] safe RL Characteristics



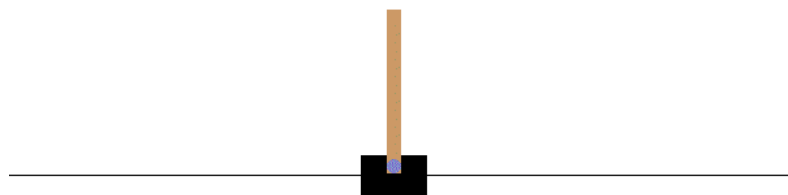
Reward signal and safety signal may contradict to each other



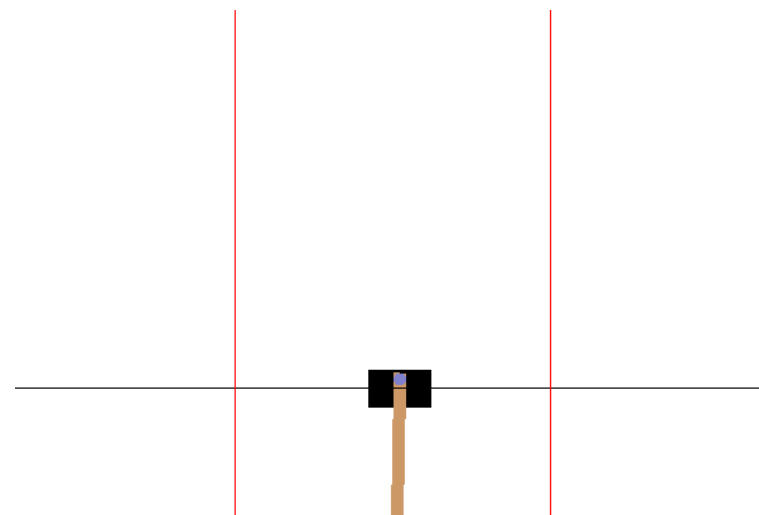
The Trajectory should consider the safety constraints

Introduction

- Safe Reinforcement Learning Task
 - Challenges



(Unconstrained) optimal control problem [3]



Constrained optimal control problem [4]



Non-trivial!

[3] https://www.gymnasium.dev/environments/classic_control/cart_pole/

[4] <https://github.com/jemaw/gym-safety>

Related Works

- Safety RL
 - Policy gradient methods [5-7]
 - Limitations: cannot guarantee strict feasibility of the policies
(even when initialized with feasible initial policies)
 - Especially, CPO[7]'s feasibility is rarely guaranteed during learning in practice
 - Errors in gradient and Hessian matrix estimation

Can we develop a reinforcement learning algorithm that explicitly addresses **the priority of constraint satisfaction?**

[5] Uchibe, E., & Doya, K. (2007, July). Constrained reinforcement learning from intrinsic and extrinsic rewards. In 2007 IEEE 6th International Conference on Development and Learning (pp. 163-168). IEEE.

[6] Chow, Y., Ghavamzadeh, M., Janson, L., & Pavone, M. (2018). Risk-constrained reinforcement learning with percentile risk criteria. Journal of Machine Learning Research, 18(167), 1-51.

[7] Achiam, J., Held, D., Tamar, A., & Abbeel, P. (2017, July). Constrained policy optimization. In International conference on machine learning (pp. 22-31). PMLR.

Related Works

- Cross Entropy Method

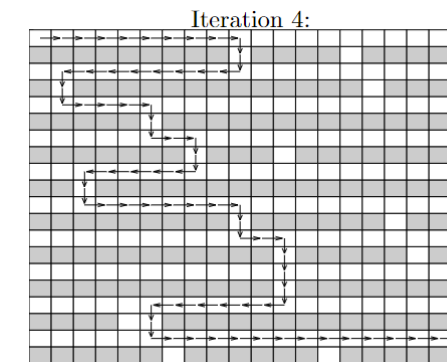
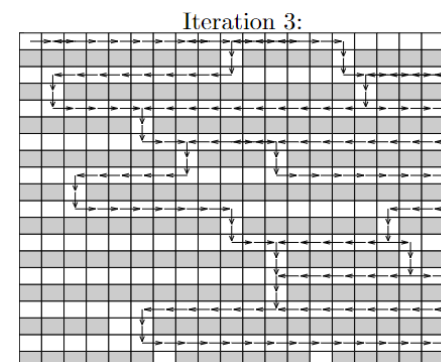
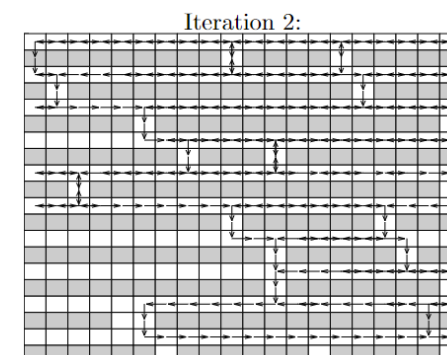
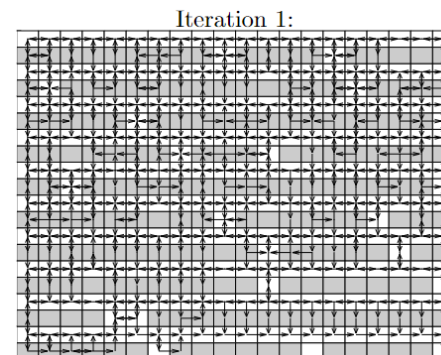
Algorithm 2.1 The CE Method for Stochastic Optimization

1. Choose some $\hat{\mathbf{v}}_0$. Set $t = 1$ (level counter).
2. Generate a sample $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(N)}$ from the density $f(\cdot; \mathbf{v}_{t-1})$ and compute the sample $(1 - \rho)100\%$ -percentile $\hat{\gamma}_t$ of the sample scores.
3. Use the **same** sample $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(N)}$ and solve the stochastic program (3). Denote the solution by $\tilde{\mathbf{v}}_t$.
4. Apply (4) to smooth out the vector $\tilde{\mathbf{v}}_t$.
5. If for some $t \geq d$, say $d = 5$, $\hat{\gamma}_t = \hat{\gamma}_{t-1} = \dots = \hat{\gamma}_{t-d}$ then **stop** (let T denote the final iteration); otherwise set $t = t + 1$ and reiterate from step 2.

The CE method for stochastic optimization [8]

$$(3) \max_{\mathbf{v}} \hat{D}(\mathbf{v}) = \max_{\mathbf{v}} \frac{1}{N} \sum_{i=1}^N \mathbf{I}_{\{s(\mathbf{X}^{(i)}) \geq \hat{\gamma}_t\}} \log f(\mathbf{X}^{(i)}; \mathbf{v})$$

$$(4) \hat{\mathbf{v}}_t = \alpha \tilde{\mathbf{v}}_t + (1 - \alpha) \hat{\mathbf{v}}_{t-1},$$



Results for the 20 x 20 maze [8]

Related Works

- CEM with constraints
 - [9-10] discard violated trajectories
 - Can be considered as a special case of our work when the constraint function has binary outputs

[9] Kobilarov, M. (2012). Cross-entropy randomized motion planning.

[10] Livingston, S. C., Wolff, E. M., & Murray, R. M. (2015, April). Cross-entropy temporal logic motion planning. In Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control (pp. 269-278).

Method

- Problem Formulation
 - *Constrained cross-entropy method* generalizes the cross-entropy method for unconstrained optimization [8].
 - Basic idea: generate a sequence of policy distributions that eventually concentrates on a feasible (locally) optimal policy.

Method

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family $F_{\mathcal{V}}$.

- 1: $l \leftarrow 1$. Initialize $n_l, \mathbf{v}_l, \rho, \lambda_l, \alpha_l$. $k_l \leftarrow \lceil \rho n_l \rceil$. $\hat{\eta}_l \leftarrow \mathbf{0}$.
 - 2: **repeat**
 - 3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{\mathbf{v}_l}$ i.i.d..
 - 4: **for** $i = 1, \dots, n_l$ **do**
 - 5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i}), H(\pi_{\theta_i})$.
 - 6: **end for**
 - 7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of H . Let Λ_l be the first k_l elements.
 - 8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
 - 9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of G . Let Λ_l be the first k_l elements.
 - 10: **end if**
 - 11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) \left(\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l \right)$.
 - 12: $\mathbf{v}_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
 - 13: Update n_l, λ_l, α_l . $l \leftarrow l + 1$. $k_l \leftarrow \lceil \rho n_l \rceil$.
 - 14: **until** Stopping rule is satisfied.
-

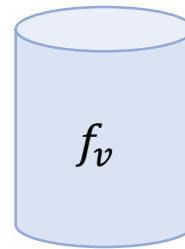
Method

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family F_V .

1: $l \leftarrow 1$. Initialize $n_l, v_l, \rho, \lambda_l, \alpha_l$. $k_l \leftarrow \lceil \rho n_l \rceil$. $\hat{\eta}_l \leftarrow \mathbf{0}$.
2: **repeat**
3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$ i.i.d..
4: **for** $i = 1, \dots, n_l$ **do**
5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i}), H(\pi_{\theta_i})$.
6: **end for**
7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of H . Let Λ_l be the first k_l elements.
8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of G . Let Λ_l be the first k_l elements.
10: **end if**
11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) \left(\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l \right)$.
12: $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
13: Update n_l, λ_l, α_l . $l \leftarrow l + 1$. $k_l \leftarrow \lceil \rho n_l \rceil$.
14: **until** Stopping rule is satisfied.

- (step 3)
 - Sample policies θ over the policy distribution f_v



Method

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family F_V .

1: $l \leftarrow 1$. Initialize $n_l, v_l, \rho, \lambda_l, \alpha_l$. $k_l \leftarrow \lceil \rho n_l \rceil$. $\hat{\eta}_l \leftarrow \mathbf{0}$.
2: **repeat**
3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$ i.i.d..
4: **for** $i = 1, \dots, n_l$ **do**
5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i}), H(\pi_{\theta_i})$.
6: **end for**
7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of H . Let Λ_l be the first k_l elements.
8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of G . Let Λ_l be the first k_l elements.
10: **end if**
11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) \left(\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l \right)$.
12: $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
13: Update n_l, λ_l, α_l . $l \leftarrow l + 1$. $k_l \leftarrow \lceil \rho n_l \rceil$.
14: **until** Stopping rule is satisfied.

- (step 5)

- Evaluate their G-values and H-values

$$G_J(\pi) := \mathbb{E}_{\tau \sim P_\pi} [J(\tau)]$$

$J: (S \times A)^N \rightarrow \mathbb{R}$: an objective function

$$H_Z(\pi) := \mathbb{E}_{\tau \sim P_\pi} [Z(\tau)]$$

$Z: (S \times A)^N \rightarrow \mathbb{R}$: a cost function

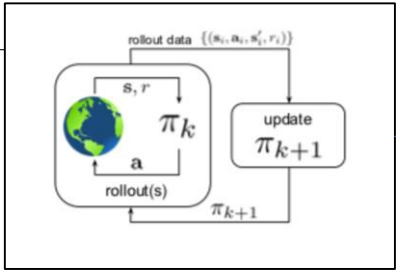
$$\tau = s_0, a_0, \dots, s_{N-1}, a_{N-1}$$

Method

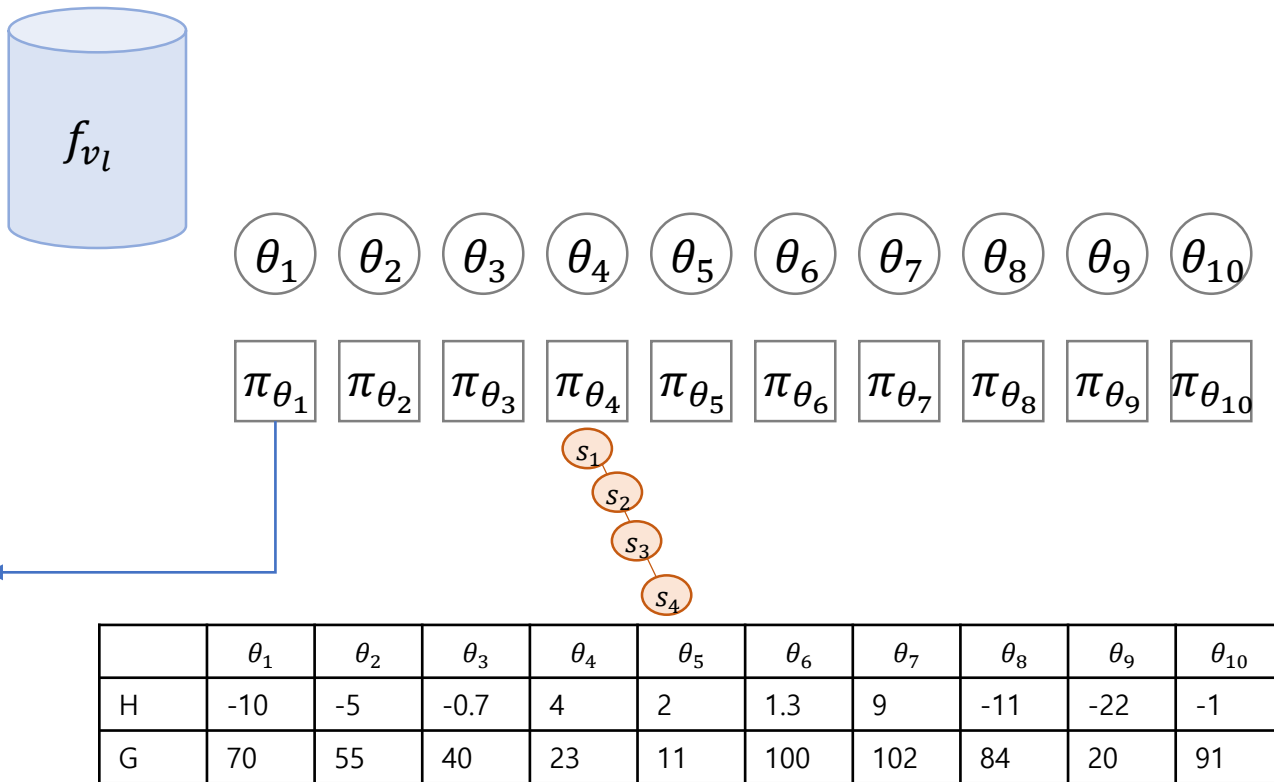
Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family F_V .

- 1: $l \leftarrow 1$. Initialize $n_l, v_l, \rho, \lambda_l, \alpha_l, k_l \leftarrow \lceil \rho n_l \rceil, \hat{\eta}_l \leftarrow \mathbf{0}$.
- 2: **repeat**
- 3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$ i.i.d..
- 4: **for** $i = 1, \dots, n_l$ **do**
- 5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i}), H(\pi_{\theta_i})$.
- 6: **end for**
- 7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of H . Let Λ_l be the first k_l elements.
- 8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
- 9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of G . Let Λ_l be the first k_l elements.
- 10: **end if**
- 11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) \left(\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l \right)$.
- 12: $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
- 13: Update $n_l, \lambda_l, \alpha_l, l \leftarrow l + 1, k_l \leftarrow \lceil \rho n_l \rceil$.
- 14: **until** Stopping rule is satisfied.



- (step 5)
 - Evaluate their G-values and H-values



Method

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family F_V .

```

1:  $l \leftarrow 1$ . Initialize  $n_l, v_l, \rho, \lambda_l, \alpha_l, k_l \leftarrow \lceil \rho n_l \rceil, \hat{\eta}_l \leftarrow \mathbf{0}$ .
2: repeat
3:   Sample  $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$  i.i.d..
4:   for  $i = 1, \dots, n_l$  do
5:     Simulate  $\pi_{\theta_i}$  and estimate  $G(\pi_{\theta_i}), H(\pi_{\theta_i})$ .
6:   end for
7:   Sort  $\{\theta_i\}_{i=1}^{n_l}$  in ascending order of  $H$ . Let  $\Lambda_l$  be the first  $k_l$  elements.
8:   if  $H(\pi_{\theta_{k_l}}) \leq d$  then
9:     Sort  $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$  in descending order of  $G$ . Let  $\Lambda_l$  be the first  $k_l$  elements.
10:  end if
11:   $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) (\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l)$ .
12:   $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$ .
13:  Update  $n_l, \lambda_l, \alpha_l, l \leftarrow l + 1, k_l \leftarrow \lceil \rho n_l \rceil$ .
14: until Stopping rule is satisfied.
```

ρ -percentile of H for the current policy distribution

$$S(\pi_\theta, v, \rho) := \delta(\xi_H(\rho, v) > d) \delta(H(\pi_\theta) \leq \xi_H(\rho, v)) + \delta(\xi_H(\rho, v) \leq d) \delta(H(\pi_\theta) \leq d) \delta(U(\pi_\theta) \geq \xi_U(1 - \rho, v)),$$

$$U(\pi_\theta) := G(\pi_\theta) \delta(H(\pi_\theta) \leq d)$$

- (step 7-10)
 - Estimate $S(\pi, v, \rho)$
 - $\rho=0.2 \Rightarrow k_l=2$
 - $d=-15$

	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9	θ_{10}
H	-10	-5	-0.7	4	2	1.3	9	-11	-22	-1
G	70	55	40	23	11	100	102	84	20	91



Sort in ascending order of H

	θ_9	θ_8	θ_1	θ_2	θ_{10}	θ_3	θ_6	θ_5	θ_4	θ_7
H	-22	-11	-10	-5	-1	-0.7	1.3	2	4	9
G	20	84	70	55	91	40	100	11	23	102



π_{k_l}

$$H(\pi_{k_l}) = -11 \geq -15$$

$$\Lambda_l = \{\theta_9, \theta_8\}$$

Method

```
def _select_elites(self, rollouts: Rollouts) -> Rollouts:
    """Returns the elite rollouts.
```

If there are sufficient rollouts which satisfy the constraints, return these sorted by objective cost.
Otherwise, return rollouts sorted by constraint cost.

```
"""
```

```
feasible_ids = (rollouts.constraint_costs == 0).nonzero()
if feasible_ids.size(0) >= self._num_elites:
    _, sorted_ids_of_feasible_ids = rollouts.objective_costs[feasible_ids].squeeze().sort()
    elites_ids = feasible_ids[sorted_ids_of_feasible_ids[0:self._num_elites]].squeeze()
    return Rollouts(rollouts.trajectories[elites_ids], rollouts.actions[elites_ids],
                    rollouts.objective_costs[elites_ids], rollouts.constraint_costs[elites_ids])
else:
    _, sorted_ids = rollouts.constraint_costs.sort()
    elites_ids = sorted_ids[0:self._num_elites]
    return Rollouts(rollouts.trajectories[elites_ids], rollouts.actions[elites_ids],
                    rollouts.objective_costs[elites_ids], rollouts.constraint_costs[elites_ids])
```

Algorithm 1 Constrained Cross-Entropy

Require: An objective function G , a constrained objective function U , a NEF H .

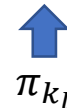
- 1: $l \leftarrow 1$. Initialize $n_l, v_l, \rho, \lambda_l, \alpha_l, k_l \leftarrow 1$.
- 2: **repeat**
- 3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$ i.i.d..
- 4: **for** $i = 1, \dots, n_l$ **do**
- 5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i})$.
- 6: **end for**
- 7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of $H(\pi_{\theta_i})$.
- 8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
- 9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of $G(\pi_{\theta_i})$.
- 10: **end if**
- 11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_{\theta})}{\sum_{\theta \in \Lambda_l} G(\pi_{\theta})} \Gamma(\theta)$
- 12: $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
- 13: Update $n_l, \lambda_l, \alpha_l, l \leftarrow l + 1, k_l \leftarrow \lfloor \rho n_l \rfloor$.
- 14: **until** Stopping rule is satisfied.

θ_7	θ_8	θ_9	θ_{10}
9	-11	-22	-1
102	84	20	91

ascending order of H

θ_6	θ_5	θ_4	θ_7
1.3	2	4	9
100	11	23	102

0	20	04	70	55	91	40
---	----	----	----	----	----	----



π_{k_l}

$$H(\pi_{k_l}) = -11 \leq -5$$

then step 9 for $\{\theta_i \mid H(\pi_{\theta_i}) \leq d\}$

ρ -percentile of H for the current policy distribution

$$S(\pi_{\theta}, v, \rho) := \delta(\xi_H(\rho, v) > d) \delta(H(\pi_{\theta}) \leq \xi_H(\rho, v)) + \delta(\xi_H(\rho, v) \leq d) \delta(H(\pi_{\theta}) \leq d) \delta(U(\pi_{\theta}) \geq \xi_U(1 - \rho, v)),$$

$$U(\pi_{\theta}) := G(\pi_{\theta}) \delta(H(\pi_{\theta}) \leq d)$$

Method

From the unofficial code [11]

```
def _select_elites(self, rollouts: Rollouts) -> Rollouts:
    """Returns the elite rollouts.

    If there are sufficient rollouts which satisfy the constraints, return these sorted by objective cost.
    Otherwise, return rollouts sorted by constraint cost.
    """
    feasible_ids = (rollouts.constraint_costs == 0).nonzero()
    if feasible_ids.size(0) >= self._num_elites:
        _, sorted_ids_of_feasible_ids = rollouts.objective_costs[feasible_ids].squeeze().sort()
        elites_ids = feasible_ids[sorted_ids_of_feasible_ids[0:self._num_elites]].squeeze()
        return Rollouts(rollouts.trajectories[elites_ids], rollouts.actions[elites_ids],
                        rollouts.objective_costs[elites_ids], rollouts.constraint_costs[elites_ids])
    else:
        _, sorted_ids = rollouts.constraint_costs.sort()
        elites_ids = sorted_ids[0:self._num_elites]
        return Rollouts(rollouts.trajectories[elites_ids], rollouts.actions[elites_ids],
                        rollouts.objective_costs[elites_ids], rollouts.constraint_costs[elites_ids])
```

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constrained policy family Π_Θ , an NEF family F_V .

- 1: $l \leftarrow 1$. Initialize $n_l, v_l, \rho, \lambda_l, \alpha_l, k_l \leftarrow \lceil \rho n_l \rceil, \hat{\eta}_l \leftarrow \mathbf{0}$.
- 2: **repeat**
- 3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$ i.i.d..
- 4: **for** $i = 1, \dots, n_l$ **do**
- 5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i}), H(\pi_{\theta_i})$.
- 6: **end for**
- 7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of H . Let Λ_l be the first k_l elements.
- 8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
- 9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of G . Let Λ_l be the first k_l elements.
- 10: **end if**
- 11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{\pi(\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) (\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l)$.
- 12: $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
- 13: Update $n_l, \lambda_l, \alpha_l, l \leftarrow l + 1, k_l \leftarrow \lceil \rho n_l \rceil$.
- 14: **until** Stopping rule is satisfied.

	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9	θ_{10}
H	-10	-5	-0.7	4	2	1.3	9	-11	-22	-1
G	70	55	40	23	11	100	102	84	20	91



Sort in ascending order of H

	θ_9	θ_8	θ_1	θ_2	θ_{10}	θ_3	θ_6	θ_5	θ_4	θ_7
H	-22	-11	-10	-5	-1	-0.7	1.3	2	4	9
G	20	84	70	55	91	40	100	11	23	102



π_{k_l}

$$H(\pi_{k_l}) = -11 \leq -5$$

then step 9 for $\{\theta_i \mid H(\pi_{\theta_i}) \leq d\}$

ρ -percentile of H for the current policy distribution

$$S(\pi_\theta, v, \rho) := \delta(\xi_H(\rho, v) > d) \delta(H(\pi_\theta) \leq \xi_H(\rho, v)) + \delta(\xi_H(\rho, v) \leq d) \delta(H(\pi_\theta) \leq d) \delta(U(\pi_\theta) \geq \xi_U(1 - \rho, v)),$$

$$U(\pi_\theta) := G(\pi_\theta) \delta(H(\pi_\theta) \leq d)$$

Method

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family F_V .

```

1:  $l \leftarrow 1$ . Initialize  $n_l, v_l, \rho, \lambda_l, \alpha_l, k_l \leftarrow \lceil \rho n_l \rceil, \hat{\eta}_l \leftarrow \mathbf{0}$ .
2: repeat
3:   Sample  $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$  i.i.d..
4:   for  $i = 1, \dots, n_l$  do
5:     Simulate  $\pi_{\theta_i}$  and estimate  $G(\pi_{\theta_i}), H(\pi_{\theta_i})$ .
6:   end for
7:   Sort  $\{\theta_i\}_{i=1}^{n_l}$  in ascending order of  $H$ . Let  $\Lambda_l$  be the first  $k_l$  elements.
8:   if  $H(\pi_{\theta_{k_l}}) \leq d$  then
9:     Sort  $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$  in descending order of  $G$ . Let  $\Lambda_l$  be the first  $k_l$  elements.
10:  end if
11:   $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) (\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l)$ .
12:   $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$ .
13:  Update  $n_l, \lambda_l, \alpha_l, l \leftarrow l + 1, k_l \leftarrow \lceil \rho n_l \rceil$ .
14: until Stopping rule is satisfied.
```


 π_{k_l}

• (step 7-10)

- Estimate $S(\pi, v, \rho)$
- $\rho=0.2 \Rightarrow k_l=2$
- $d=-5$

	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9	θ_{10}
H	-10	-5	-0.7	4	2	1.3	9	-11	-22	-1
G	70	55	40	23	11	100	102	84	20	91



Sort in ascending order of H

	θ_9	θ_8	θ_1	θ_2	θ_{10}	θ_3	θ_6	θ_5	θ_4	θ_7
H	-22	-11	-10	-5	-1	-0.7	1.3	2	4	9
G	20	84	70	55	91	40	100	11	23	102



Sort in descending order of G

	θ_8	θ_1	θ_9
H	-11	-10	-22
G	84	70	20

$\Lambda_l = \{\theta_8, \theta_1\}$

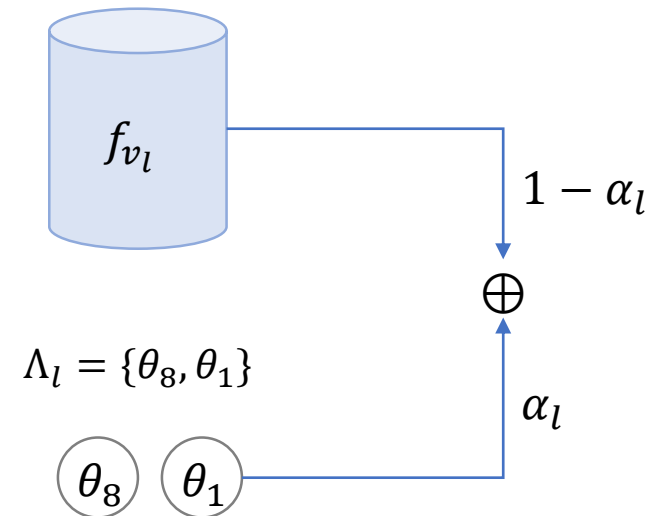
Method

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family F_V .

- 1: $l \leftarrow 1$. Initialize $n_l, v_l, \rho, \lambda_l, \alpha_l$. $k_l \leftarrow \lceil \rho n_l \rceil$. $\hat{\eta}_l \leftarrow \mathbf{0}$.
 - 2: **repeat**
 - 3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$ i.i.d..
 - 4: **for** $i = 1, \dots, n_l$ **do**
 - 5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i}), H(\pi_{\theta_i})$.
 - 6: **end for**
 - 7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of H . Let Λ_l be the first k_l elements.
 - 8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
 - 9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of G . Let Λ_l be the first k_l elements.
 - 10: **end if**
 - 11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) \left(\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l \right)$.
 - 12: $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
 - 13: Update n_l, λ_l, α_l . $l \leftarrow l + 1$. $k_l \leftarrow \lceil \rho n_l \rceil$.
 - 14: **until** Stopping rule is satisfied.
-

- (step 11)
 - Estimate $m(v_{l+1})$ with $\hat{\eta}_{l+1}$



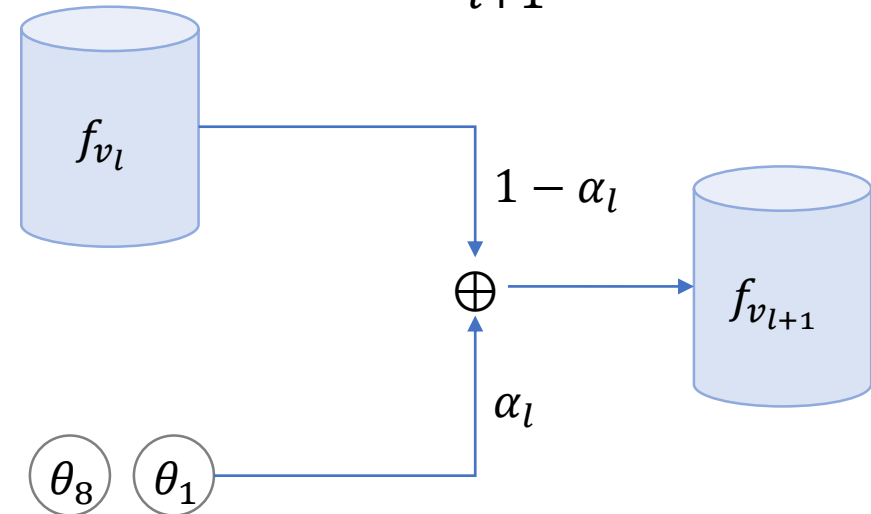
Method

Algorithm 1 Constrained Cross-Entropy Method

Require: An objective function G , a constraint function H , a constraint upper bound d , a class of parameterized policies Π_Θ , an NEF family F_V .

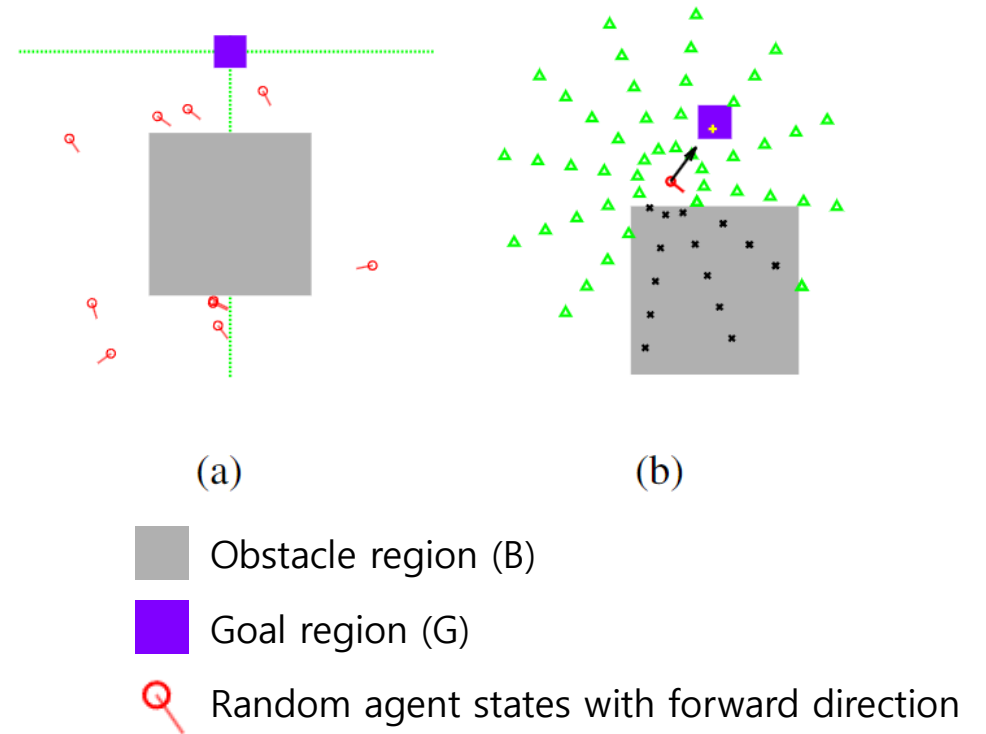
- 1: $l \leftarrow 1$. Initialize $n_l, v_l, \rho, \lambda_l, \alpha_l$. $k_l \leftarrow \lceil \rho n_l \rceil$. $\hat{\eta}_l \leftarrow \mathbf{0}$.
 - 2: **repeat**
 - 3: Sample $\theta_1, \dots, \theta_{n_l} \sim f_{v_l}$ i.i.d..
 - 4: **for** $i = 1, \dots, n_l$ **do**
 - 5: Simulate π_{θ_i} and estimate $G(\pi_{\theta_i}), H(\pi_{\theta_i})$.
 - 6: **end for**
 - 7: Sort $\{\theta_i\}_{i=1}^{n_l}$ in ascending order of H . Let Λ_l be the first k_l elements.
 - 8: **if** $H(\pi_{\theta_{k_l}}) \leq d$ **then**
 - 9: Sort $\{\theta_i \mid H(\pi_{\theta_i}) \geq d\}$ in descending order of G . Let Λ_l be the first k_l elements.
 - 10: **end if**
 - 11: $\hat{\eta}_{l+1} \leftarrow \alpha_l \sum_{\theta \in \Lambda_l} \frac{G(\pi_\theta)}{\sum_{\theta \in \Lambda_l} G(\pi_\theta)} \Gamma(\theta) + (1 - \alpha_l) \left(\frac{\lambda_l}{n_l} \sum_{i=1}^{n_l} \Gamma(\theta_i) + (1 - \lambda_l) \hat{\eta}_l \right)$.
 - 12: $v_{l+1} \leftarrow m^{-1}(\hat{\eta}_{l+1})$.
 - 13: Update n_l, λ_l, α_l . $l \leftarrow l + 1$. $k_l \leftarrow \lceil \rho n_l \rceil$.
 - 14: **until** Stopping rule is satisfied.
-

- (step 12)
 - Update the sampling distribution to v_{l+1}



Experiments

- Simulation
 - Robot navigation task
- Setting
 - State space: $S = \{(x, y, \text{zeta})\}$
 - Agent's position, orientation in the global coord.
 - action space: $A = \{(v, w)\}$
 - Linear and angular speed
- Agent policy: Fully connected neural network
 - 2 hidden layer, 30 nodes in each layer
 - Trajectory length $N=30$
 - For CCE, sample 40 different policies in each iteration
 - Each sampled policy is evaluated using 10 sample trajectories
 - Selecting elite example: $\rho = 0.2$



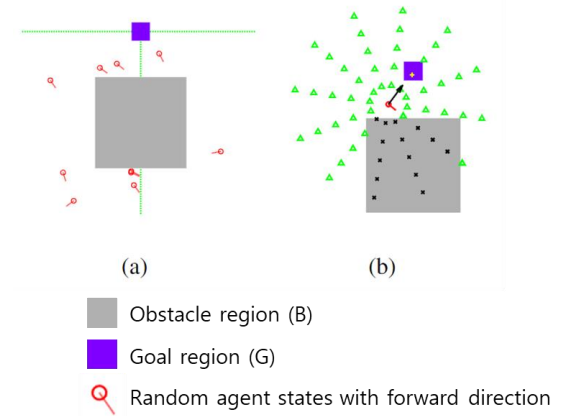
Experiments

- Configurations

Table 1: $J_i(\tau)$, $Z_i(\tau)$ and constraint upper bound d_i for $i = 1, 2, 3, 4$, $\tau \in (\mathcal{S} \times \mathcal{A})^N$.

i	$J_i(\tau)$	$Z_i(\tau)$	d_i	J_i Markovian	Z_i Markovian
1	1 for each state in $\mathcal{G}$; $2 y $ for each state with $y \in [-2, -0.2]$; 0 otherwise.	-1 if the robot arrives $\mathcal{G}$ which is absorbing; 0 otherwise.	-0.5	Yes	Yes
2	30 times the minimum signed distance from any state in τ to $\mathcal{B}$.	-1 if the robot visited $\mathcal{G}$ in τ ; 0 otherwise.	-0.5	No	No
3	Same as $J_2(\tau)$.	-1 for each state in $\mathcal{G}$; 0 otherwise.	-5	No	Yes
4	Same as $J_1(\tau)$.	-1 if the robot visits $\mathcal{G}$ and never visits $\mathcal{B}$; 0 otherwise.	-0.5	Yes	No

* Initialize with random generated **infeasible policy!**



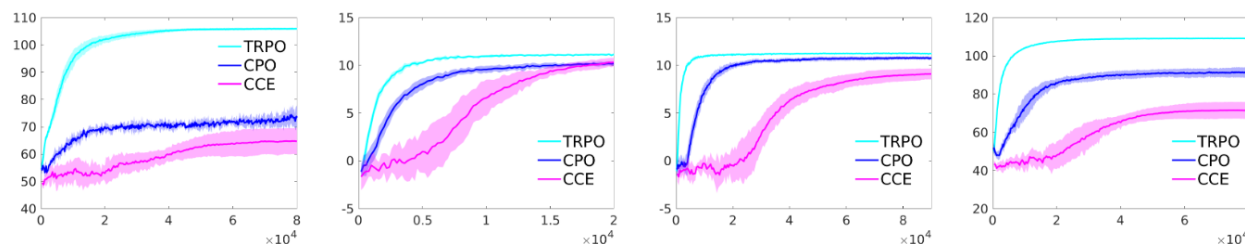
Experiments

• Results

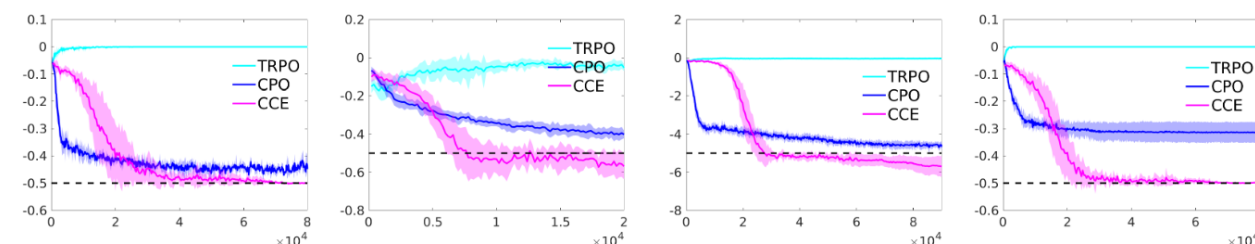
Table 1: $J_i(\tau)$, $Z_i(\tau)$ and constraint upper bound d_i for $i = 1, 2, 3, 4$, $\tau \in (\mathcal{S} \times \mathcal{A})^N$.

i	$J_i(\tau)$	$Z_i(\tau)$	d_i	J_i Markovian	Z_i Markovian
1	1 for each state in $\mathcal{G}$; $2 y $ for each state with $y \in [-2, -0.2]$; 0 otherwise.	-1 if the robot arrives $\mathcal{G}$ which is absorbing; 0 otherwise.	-0.5	Yes	Yes
2	30 times the minimum signed distance from any state in τ to $\mathcal{B}$.	-1 if the robot visited $\mathcal{G}$ in τ ; 0 otherwise.	-0.5	No	No
3	Same as $J_2(\tau)$.	-1 for each state in $\mathcal{G}$; 0 otherwise.	-5	No	Yes
4	Same as $J_1(\tau)$.	-1 if the robot visits $\mathcal{G}$ and never visits $\mathcal{B}$; 0 otherwise.	-0.5	Yes	No

Objective value G



Constraint value H



(a) $i = 1$

(b) $i = 2$

(c) $i = 3$

(d) $i = 4$

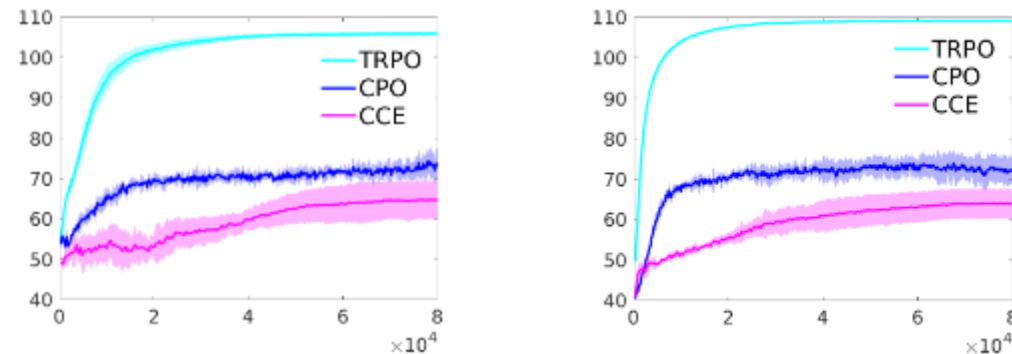
X-axis: the total number of sample trajectories
Y-axis: average objective / constraint values

- CCE successfully **outputs feasible policies** in all experiments
- CPO needs significantly more samples to find a single feasible policy, or **simply converges to an infeasible policy**

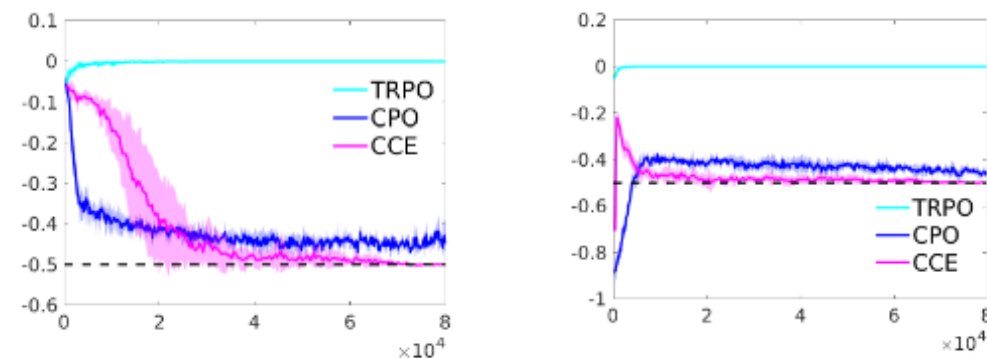
Experiments

- Results
 - initial feasible policy

Objective value G



Constraint value H



(a) init. w/ unfeasible policy

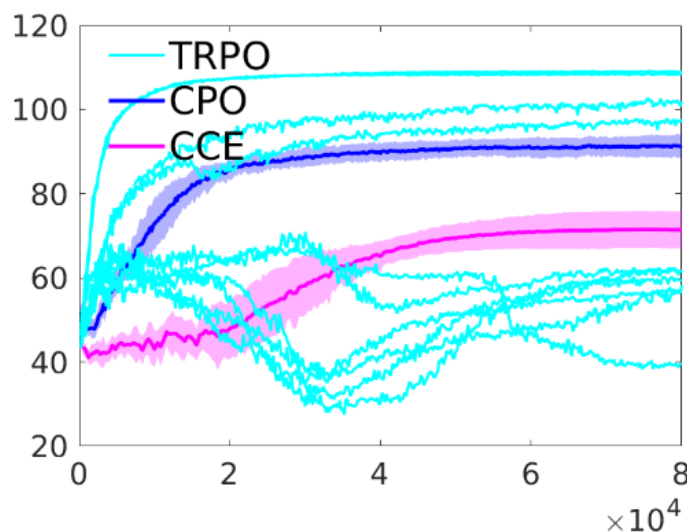
(a) init. w/ feasible policy

- CPO leaves the feasible region rapidly and then follows generally the same path as if it is initialized with an infeasible policy
 - CPO's incapability to enforce constrain satisfaction is **not due to the lack of initial feasibility**.
- CCE also leaves the feasible region at an early stage of iterations, but regains feasibility much faster than the infeasible initial policies
 - CCE is **more reliable than CPO** when the strict constraint satisfaction is critical.

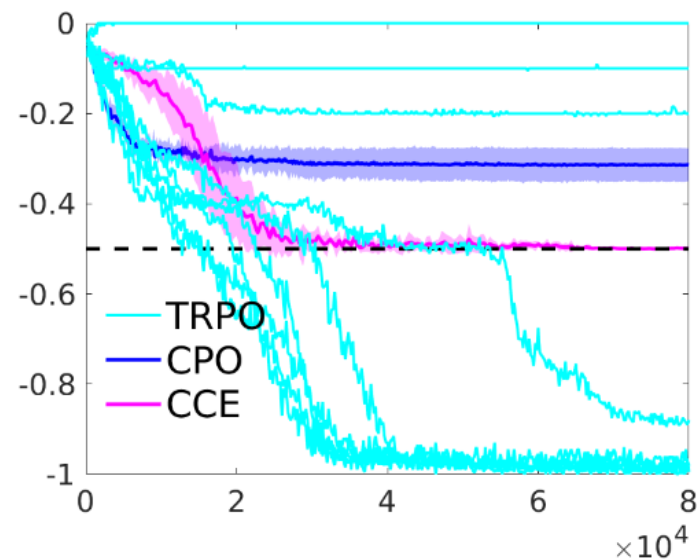
Experiments

- Results

- Exp 4
- TRPO with objective G-100H



(a) $G_{J_4}(\pi_\theta)$.



(b) $H_{Z_4}(\pi_\theta)$.

- With a fixed penalty coefficient, the TRPO policies are either **infeasible** or with **very small constraint values**

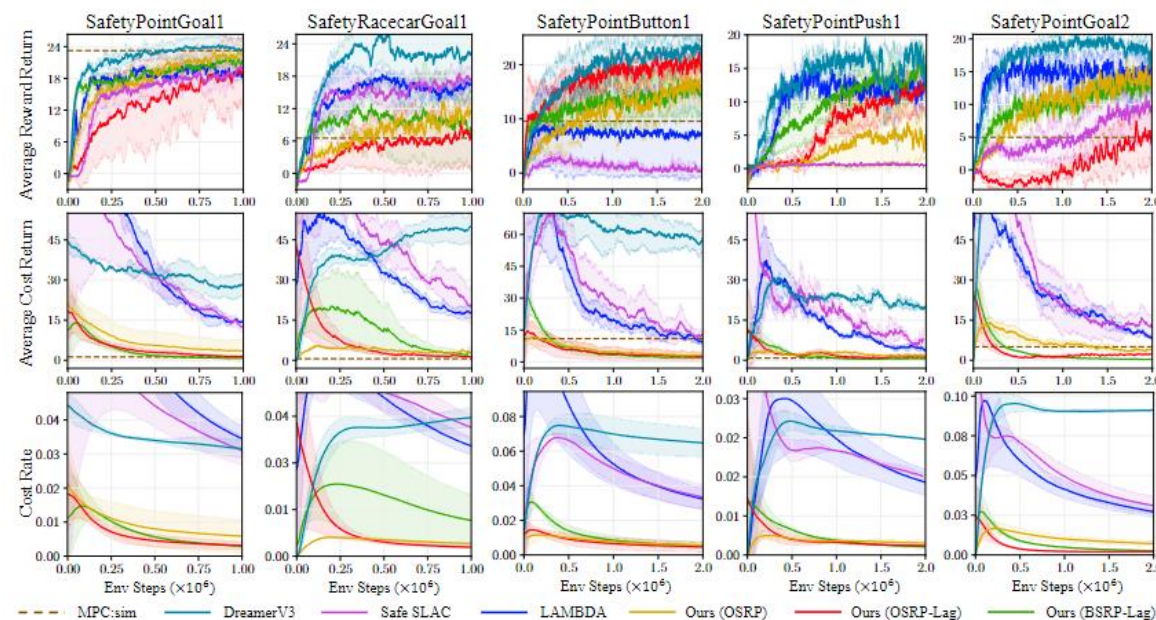
Conclusion

- Proposed a **constrained cross-entropy-based method** to solve safe reinforcement learning problem.
- Analyzed its **asymptotic performance** using an ODE, proved its **convergence**.
- Showed with simulation experiments that our method can effectively **learn feasible policies without assumptions**.

Applications

- In SafeDreamer

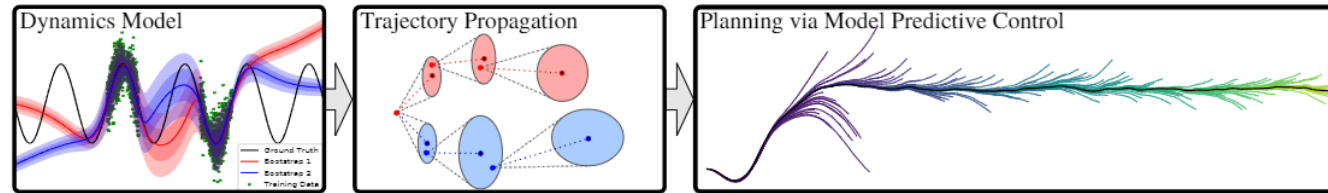
- (1) OSRP (Online Safety-Reward Planning)
 - Use Planner with CEM
- (2) BSRP (Background Safety-Reward Planning)
 - Use Actor
- (3) MPC:sim
 - Use Planner with CCEM / RCE



Comparing SafeDreamer to model-based baselines

PETS

- Probabilistic Models (PE) + Trajectory Sampling (TS)
 - Uses CEM in the model-based RL
 - Unconstrained RL



Algorithm 1 Our model-based MPC algorithm ‘PETS’:

- 1: Initialize data $\mathbb{D}$ with a random controller for one trial.
 - 2: **for** Trial $k = 1$ to K **do**
 - 3: Train a *PE* dynamics model $\tilde{f}$ given $\mathbb{D}$.
 - 4: **for** Time $t = 0$ to TaskHorizon **do**
 - 5: **for** Actions sampled $\mathbf{a}_{t:t+T} \sim \text{CEM}(\cdot)$, 1 to NSamples **do**
 - 6: Propagate state particles $\mathbf{s}_\tau^p$ using *TS* and $\tilde{f}|\{\mathbb{D}, \mathbf{a}_{t:t+T}\}$.
 - 7: Evaluate actions as $\sum_{\tau=t}^{t+T-1} \frac{1}{P} \sum_{p=1}^P r(\mathbf{s}_\tau^p, \mathbf{a}_\tau)$
 - 8: Update $\text{CEM}(\cdot)$ distribution.
 - 9: Execute first action $\mathbf{a}_t^*$ (only) from optimal actions $\mathbf{a}_{t:t+T}^*$.
 - 10: Record outcome: $\mathbb{D} \leftarrow \mathbb{D} \cup \{\mathbf{s}_t, \mathbf{a}_t^*, \mathbf{s}_{t+1}\}$.
-

Sample trajectories
Evaluates and make elite sample distributions
Update policy distribution

Robust Cross Entropy

- Constrained RL
- Similar to the CCEM, but it differs from
 - (1) RCE consider the worst-case cost , aim to minimize the maximum cost to select the feasible set
 - CCEM calculate the expectation
 - (2) RCE optimizes the action sequence within the planning horizon in the model-based RL setting
 - CCEM is to optimize the policy parameters for model-free RL

Algorithm 2 MPC with RCE

Input: Initial collected data $\mathcal{D}$; RCE parameters $\mathcal{P}$

```

1: while The performance is not converged do
2:   Train the dynamics  $\bar{f}$  and cost model  $\bar{c}$  given  $\mathcal{D}$ 
3:   for Time  $t = 0$  to EpisodeLength do
4:     Observe state  $s_t$  from the environment
5:     Optimize actions by Alg. 1:
        $\{a_i^*\}_{i=t}^{t+T} \leftarrow \text{RCE}(\mathcal{P}, s_t)$ 
6:     Apply the first action  $a_t^*$  in  $\{a_i^*\}_{i=t}^{t+T}$  to the system

7:   Observe next state  $s_{t+1}$  and cost signal  $c(s_{t+1})$ 
8:   Update data buffer:
        $\mathcal{D} \leftarrow \mathcal{D} \cup \{s_t, a_t, s_{t+1}, c(s_{t+1})\}$ 
9:   end for
10: end while
    
```

Algorithm 1 Robust Cross-Entropy Method for RL

Input: Initial distribution parameter Θ ; number of samples N ; number of elites k ; initial state s_0

Output: Sample $\mathcal{X}^*$ with the highest reward

```

1: while The stop criteria is not satisfied do
2:   Draw  $N$  samples from the initial distribution:
        $\mathcal{X}_1, \mathcal{X}_2, \dots, \mathcal{X}_N \sim \mathcal{N}(\Theta)$ 
3:   Evaluate each sample  $\mathcal{X}_i$  by Eq.2 to get the estimation
       of reward  $r(\mathcal{X}_i; s_0)$  and cost  $c(\mathcal{X}_i; s_0)$ 
4:   Select the feasible set  $\Omega \in \{\mathcal{X}_i\}_{i=1}^N$  based on the cost
       estimation
5:   if  $\Omega$  is empty then
6:     Sort  $\{\mathcal{X}_i\}_{i=1}^N$  in ascending order w.r.t the cost. Let
        $\Lambda_k$  be the first  $k$  elements
7:   else
8:     Sort  $\Omega$  in descending order w.r.t the reward.
9:     Let  $\Lambda_k$  be the first  $k$  elements of  $\Omega$  if  $|\Omega| > k$ ,
       otherwise let  $\Lambda_k$  be  $\Omega$ 
10:  end if
11:  Update  $\Theta$  by maximizing the likelihood given  $\Lambda_k$ :
        $\Theta \leftarrow \arg \max_{\theta} \prod_{\mathcal{X} \in \Lambda_k} p(\mathcal{X}; \theta)$ 
12: end while
13: return  $\mathcal{X}^*$  with highest reward in  $\Lambda_k$ 
    
```

Sample trajectories

Evaluates and
make elite sample distributions

Update policy distribution

PlaNet

- MPC With the World Model
 - Uses the learned world model as the dynamics model

Algorithm 1: Deep Planning Network (PlaNet)

Input:

R Action repeat $p(s_t | s_{t-1}, a_{t-1})$ Transition model
 S Seed episodes $p(o_t | s_t)$ Observation model
 C Collect interval $p(r_t | s_t)$ Reward model
 B Batch size $q(s_t | o_{\leq t}, a_{< t})$ Encoder
 L Chunk length $p(\epsilon)$ Exploration noise
 α Learning rate

Initialize dataset $\mathcal{D}$ with S random seed episodes.

Initialize model parameters θ randomly.

while not converged do

// Model fitting

for update step $s = 1..C$ **do**

Draw sequence chunks $\{(o_t, a_t, r_t)_{t=k}^{L+k}\}_{i=1}^B \sim \mathcal{D}$
 uniformly at random from the dataset.
 Compute loss $\mathcal{L}(\theta)$ from Equation 3.
 Update model parameters $\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$.

// Data collection

$o_1 \leftarrow \text{env.reset}()$

for time step $t = 1..\lceil \frac{T}{R} \rceil$ **do**

Infer belief over current state $q(s_t | o_{\leq t}, a_{< t})$ from the history.

$a_t \leftarrow \text{planner}(q(s_t | o_{\leq t}, a_{< t}), p)$, see
 Algorithm 2 in the appendix for details.

Add exploration noise $\epsilon \sim p(\epsilon)$ to the action.

for action repeat $k = 1..R$ **do**

$r_t^k, o_{t+1}^k \leftarrow \text{env.step}(a_t)$

$r_t, o_{t+1} \leftarrow \sum_{k=1}^R r_t^k, o_{t+1}^k$

$\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_t, a_t, r_t)_{t=1}^T\}$

Algorithm 2: Latent planning with CEM

Input: H Planning horizon distance $q(s_t | o_{\leq t}, a_{< t})$ Current state belief
 I Optimization iterations $p(s_t | s_{t-1}, a_{t-1})$ Transition model
 J Candidates per iteration $p(r_t | s_t)$ Reward model
 K Number of top candidates to fit

Initialize factorized belief over action sequences $q(a_{t:t+H}) \leftarrow \text{Normal}(0, \mathbb{I})$.

for optimization iteration $i = 1..I$ **do**

// Evaluate J action sequences from the current belief.

for candidate action sequence $j = 1..J$ **do**

$a_{t:t+H}^{(j)} \sim q(a_{t:t+H})$

$s_{t:t+H+1}^{(j)} \sim q(s_t | o_{1:t}, a_{1:t-1}) \prod_{\tau=t+1}^{t+H+1} p(s_{\tau} | s_{\tau-1}, a_{\tau-1}^{(j)})$

Sample trajectories

$R^{(j)} = \sum_{\tau=t+1}^{t+H+1} \mathbb{E}[p(r_{\tau} | s_{\tau}^{(j)})]$

Evaluates and make elite sample distributions
best action sequences.

// Re-fit belief to the K best action sequences.

$\mathcal{K} \leftarrow \text{argsort}(\{R^{(j)}\}_{j=1}^J)_{1:K}$

$\mu_{t:t+H} = \frac{1}{K} \sum_{k \in \mathcal{K}} a_{t:t+H}^{(k)}, \sigma_{t:t+H} = \frac{1}{K-1} \sum_{k \in \mathcal{K}} |a_{t:t+H}^{(k)} - \mu_{t:t+H}|$

Update policy distribution

$q(a_{t:t+H}) \leftarrow \text{Normal}(\mu_{t:t+H}, \sigma_{t:t+H}^2 \mathbb{I})$

return first action mean μ_t .

SafeDreamer

- Online Safety-Reward Planning via World Models

Algorithm 2: SafeDreamer. Color green is OSRP-Lag or OSRP, purple is BSRP-Lag

Input: batch length T , batch size B , episode length L , initial Lagrangian multiplier λ_p

- 1 Initialize the world models parameters ϕ , actor-critic parameters $\theta, V_{\psi_r}, V_{\psi_c}$;
- 2 Initialize dataset $\mathcal{D}$ using a random policy;
- 3 **while** not converged **do**
- 4 Sample B trajectories $\{o_t, a_t, o_{t+1}, r_{t+1}, c_{t+1}\}_{t:t+T} \sim \mathcal{D}$;
- 5 Update the world models via minimize Equation (11);
- 6 Condense $o_{t:t+T}$ into $s_{t:t+T}$ via world models;
- 7 Generate latent rollouts using the actor within the world model with $s_{t:t+T}$ as the initial state;
- 8 Update the reward and cost critics via minimize Equation (13);
- 9 Update the safe actor and λ_p via Equation (9) and Equation (10); // Only BSRP-Lag
- 10 $\mathcal{J}_C \leftarrow 0$;
- 11 **for** $t = 1$ to L **do**
- 12 Condense o_t into latent state via $s_t \sim S_\phi(o_t, s_{t-1}, a_{t-1})$;
- 13 Sample $a_t \sim \pi_a(\cdot|s_t)$; // Only BSRP-Lag
- 14 Sample $a_t \sim \text{Planner}(s_t, \lambda_p)$; // Only OSRP-Lag or OSRP
- 15 Execute action a_t , observe $o_{t+1}, r_{t+1}, c_{t+1}$ returned from the environment;
- 16 Update dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{o_t, a_t, o_{t+1}, r_{t+1}, c_{t+1}\}$;
- 17 $\mathcal{J}_C \leftarrow \mathcal{J}_C + c_{t+1}$;
- 18 **end**
- 19 Use $\mathcal{J}_C$ to update λ_p via Algorithm 3; // Only OSRP-Lag
- 20 **end**

Algorithm 1: Online Safety-Reward Planning.
Red is OSRP only and green is OSRP-Lag only.

- 1 **Input:** current model state s_t , planning horizon H ,
- 2 num sample/policy/safe trajectories $N_{\pi_N}, N_{\pi_\theta}, N_s$,
- 3 Lagrangian multiplier λ_p , cost limit b, μ^0, σ^0 for $\mathcal{N}$
- 4 **for** $j \leftarrow 1$ to J **do**
- 5 Init. empty safe/candidate/elite actions set
 A_s, A_c, A_e
- 6 Sample $N_{\pi_N} + N_{\pi_\theta}$ traj. $\{s_i, a_i, s_{i+1}\}_{i=t}^{t+H}$
 using $\mathcal{N}(\mu^{j-1}, (\sigma^{j-1})^2 \mathbf{I}), \pi_\theta$ within S_ϕ with
 s_t as the initial state
- 7 **for all** $N_{\pi_N} + N_{\pi_\theta}$ trajectories **do**
- 8 Init. trajectory cost $J_\phi^{C,H}(\pi) \leftarrow 0$
- 9 **for** $t \leftarrow H$ to 1 **do**
- 10 Compute $R^\lambda(s_t), C^\lambda(s_t)$ via
 Equation (8)
- 11 $J_\phi^{C,H} \leftarrow \gamma J_\phi^{C,H} + C_\phi(s_t)$
- 12 **end**
- 13 $J_\phi^R, J_\phi^C \leftarrow R^\lambda(s_1), C^\lambda(s_1)$
- 14 $J_\phi^{C'} \leftarrow (J_\phi^{C,H} / H)L$
- 15 **if** $J_\phi^{C'} < b$ **then**
- 16 $A_s \leftarrow A_s \cup \{a_{t:t+H}\}$
- 17 **end**
- 18 **end**
- 19 **if** $|A_s| < N_s$ **then**
- 20 $\Omega = -J_\phi^{C'}, A_c = \{a_{t:t+H}\}_{i=1}^{N_{\pi_N} + N_{\pi_\theta}}$
- 21 **else**
- 22 $\Omega = J_\phi^R, A_c = A_s$
- 23 $\Omega = J_\phi^R - \lambda_p J_\phi^C, A_c = A_s$
- 24 **end**
- 25 Select the top-k action trajectories with highest
 Ω values among A_c as elite actions A_e
- 26 $\mu^j, \sigma^j = \text{MEAN}(A_e), \text{STD}(A_e)$
- 27 **end**
- 28 **Return:** $a \sim \mathcal{N}(\mu^j, (\sigma^j)^2 \mathbf{I})$

Sample trajectories

Evaluates and make elite sample distributions

Update policy distribution

Current Work

Current Work

- Applied KAIST EE
 - Status: 서류 제출 완료

- Timeline
 - ~~7. 10. (수) 원서접수~~
 - ~~7. 12. (금) 18:00 서류 제출~~
 - 8. 8. (목) 14:00 1차 합격자 발표
 - 8. 13. (화) 면접
 - 9. 26. (목) 14:00 최종 합격자 발표



지원서작성마이페이지제출서류관리합격자발표

Home > 제출서류관리



등록서약 / 서류제출

KAIST, 세계의 중심에서! 심 없는 노력과 끝없는 도전이 계속되는 KAIST는 열정입니다.

※ 제출서류를 확인하시고 기한내 제출 바랍니다.
[서류제출기간: 2024-06-28 10:00 ~ 2024-07-12 18:00]

▪ 지원자 정보

전월구분	2025 학년도 봄학기 1차	지원과정	석사
지원자명	함형찬	수험번호	110959
지원분야	전기및전자공학부	학생구분	• 1선택 : 국비 또는 KAIST

최종제출여부

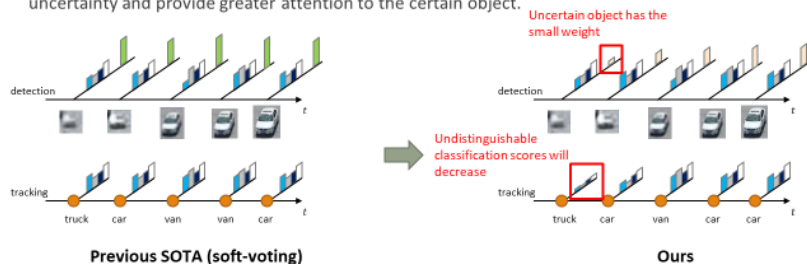
도착(접수완료)

Current Work

- Group Meeting Presentation
 - 7. 18. (Thu) 16:00 ~ 18:00
 - E3-2 2219호

Motivation

- The objectness score might be high even when the class of an object is indistinguishable.
- To predict the class of the tracking object, we need to take less into account objects with lower levels of uncertainty and provide greater attention to the certain object.



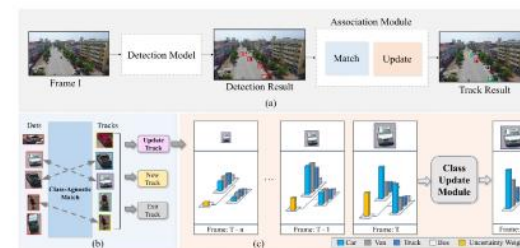
Motivation: Use the uncertainty value as the voting weight

Method

- Uncertainty-based approach
 - Calculate the uncertainty weight from the detection result. Uncertainty weight is the reciprocal of the entropy (eq. 1)
 - Accumulate the classification scores multiplied with the uncertainty weight.

Equation 1. p_{c_i} is the class probability of the class c_i

$$w = \frac{1}{E} = -\frac{1}{\sum_c p_c \log(p_c)}$$



Future Plan

Future Plan

- More survey on the OOD Autonomous Driving